Teach me about n8n

n8n (pronounced "n-eight-n") is a free and open-source node-based workflow automation tool. It allows you to connect and automate different services and systems, often with little to no code. It was designed to help automate tasks and workflows, making processes more efficient and data-driven.

Here are some key features and concepts associated with n8n:

1. **Nodes and Workflows**: In n8n, a node represents a software or a service like Gmail, Slack, or even HTTP requests. By combining nodes in different ways, you create what's known as a workflow. A workflow can represent a whole automated process like, "When I receive an email (Gmail node), post a message in Slack (Slack node)."
2. **Triggers and Actions**: Nodes can either act as triggers (starting points of workflows) or actions (steps that do something like sending emails or manipulating data).
3. **Open Source and Self-hosted**: One key aspect of n8n is that it's open source, meaning its code is freely available to use, modify, and distribute. Additionally, it's self-hosted. Instead of running on a vendor's server, you can run it on your own server. This has implications for data privacy because data stays within your control.
4. **Web-based Interface**: n8n provides a web-based interface, making it relatively easy to create and visualize workflows.
5. **Built-in OAuth and API Key Support**: n8n has built-in support for popular authorization methods, such as OAuth and API keys, making it easier to connect with different services.
6. **Extensibility**: Because it's open source, you can extend n8n with new functionality or even add nodes for services that aren't supported out of the box.
7. **Integration**: As of my last update in September 2021, n8n had support for over 200 different services and systems.

To use n8n, you need some basic knowledge about APIs, webhooks, and data formats like JSON. Depending on the complexity of the workflows and integrations, more technical expertise might be required.

I want to build my own workflow automation tool. Give me information about individual nodes and what they look like when stored in a database

Creating a workflow automation tool is a significant task, and understanding the architecture and storage of individual nodes is crucial. In the context of workflow automation tools like n8n, nodes are individual operations that interact with a particular system or service, such as a database, an API, or an email service.

Each node can have various attributes:

1. **Type**: This signifies the kind of operation or system that this node will interact with. For instance, it could be a 'HTTP Request' node, 'Gmail' node, 'MySQL' node, etc.
2. **Parameters**: These are the settings or specific details that the node requires to perform its operation. For example, an HTTP Request node might need a URL, request method (GET, POST), headers, body, etc.
3. **Credentials**: Some nodes might require credentials to access a certain service. These could include API keys, OAuth tokens, or database credentials.
4. **Inputs/Outputs**: Nodes take in data, perform an operation, and then output data. The structure of this data should be clearly defined.

When stored in a database, a node might be represented as a record with the following fields:

- **ID**: A unique identifier for the node.
- **Workflow_ID**: The identifier of the workflow that this node is part of.
- **Type**: The type of the node as described above.
- **Parameters**: This could be stored as a JSON object, allowing for flexibility in the types and number of parameters that can be stored.
- **Credentials_ID**: A reference to a separate table or data structure where credentials are securely stored.
- **Input_Schema, Output_Schema**: These fields can store the expected structure of the input and output data. This can be especially helpful in complex workflows where the output of one node is the input to another.

Remember that building a workflow automation tool involves more than just defining and storing nodes. It requires designing a system that can execute these nodes in the correct order, handle errors, manage credentials securely, and provide a user-friendly interface for creating and managing workflows.

Also, depending on the privacy, security, and scale requirements of your tool, you might consider using a NoSQL database or a distributed system. It's also important to ensure that your tool adheres to all relevant data protection and privacy regulations, especially if it's handling sensitive data or credentials.

> How can we design a system to execute them in the correct order?

Designing a system to execute nodes or tasks in the correct order is essentially about building a system to manage dependencies and control the flow of execution. This often involves building a Directed Acyclic Graph (DAG).

In a DAG, each node represents a task, and the edges between nodes represent dependencies between tasks. Directed means that the dependencies have a direction (Task A needs to be completed before Task B), and Acyclic means there are no circular dependencies.

Here's how you can design such a system:

1. **Define the Workflow**: When users create a workflow, they'll specify which tasks need to be completed and in what order. They'll also specify the dependencies between tasks. This will create a DAG where each node is a task, and the edges represent dependencies.
2. **Topological Sorting**: You'll need to implement an algorithm to sort the tasks in a way that respects the dependencies. One such algorithm is called a topological sort. It provides an order of tasks where each task comes before all the tasks it depends on.
3. **Execution Engine**: Once you have the sorted list of tasks, you'll need a system to execute those tasks in order. This system will need to handle errors, retries, logging, and possibly parallel execution if some tasks can be run concurrently.

4. **State Management**: You'll also need a way to manage the state of each task. This could be as simple as a status field in your database (e.g., pending, running, completed, failed), or it could involve more complex state management if your workflows can be paused and resumed.
5. **Error Handling**: It's also crucial to think about how to handle errors. What should happen if a task fails? Should the whole workflow fail, or should it try to continue with other tasks? You might want to give users options to customize this.
6. **Scheduling**: Depending on your use case, you might also need to support scheduling of workflows (e.g., running a workflow every day at 2am).

Building such a system requires a solid understanding of algorithms, databases, and distributed systems, and it can take considerable time and resources. You'll also need to provide a user-friendly interface for users to create and manage workflows, and you'll need to think carefully about security and data protection, especially if your workflows are handling sensitive data or credentials.

> Whats the process of updating workflows? When they are updated, do we also do versioning? If so, how does that work? Give me database models alongside the explanations

Updating workflows in a workflow automation system is a common requirement, and handling these updates effectively is crucial for maintaining the consistency and reliability of the system. Versioning is a common practice while dealing with updates, and it's especially helpful when multiple users can edit the same workflow, or when you want to track changes over time.

Here's a high-level process of how updating and versioning of workflows might work:

1. **Workflow Updates**: When a user updates a workflow (e.g., adds/removes/edits nodes or changes the order of execution), you'd typically validate and save these changes to your database. You might have an `UpdateWorkflow` API endpoint that accepts the updated workflow and user ID.
2. **Versioning**: Each time a workflow is updated, you could create a new version of that workflow. You could store the history of all versions, or you could limit the history to

a certain number of recent versions.

3. **Reverting Changes**: Storing past versions allows users to revert to a previous version if they made a mistake or if a new version has issues.

In terms of database design, one approach is to have a separate table for workflow versions. Here's a simplified version of how the tables might look:

- `workflows` table:
  - `id` : Unique identifier for the workflow.
  - `current_version_id` : The ID of the current version of this workflow. This could be a foreign key referencing the `workflow_versions` table.
  - `created_at` , `updated_at` : Timestamps indicating when the workflow was created and last updated.
  - Other metadata about the workflow, such as its name, description, owner, etc.
- `workflow_versions` table:
  - `id` : Unique identifier for this version.
  - `workflow_id` : The ID of the workflow that this version belongs to. This is a foreign key referencing the `workflows` table.
  - `version_number` : The version number of this workflow. This could simply increment each time a new version is created.
  - `nodes` : The nodes of the workflow for this version, possibly stored as a JSON object.
  - `created_at` , `updated_at` : Timestamps indicating when this version was created and last updated.

In this design, each workflow has many versions, but only one of them is the "current" version. When a user updates a workflow, you create a new version, update the `current_version_id` in the `workflows` table, and possibly remove old versions if you're limiting the version history.

This is a simplified design and might need to be adapted based on your specific needs. For example, if your workflows are complex, you might need to normalize the `nodes` into their own table rather than storing them as a JSON object. Also, handling concurrent updates can be tricky and might require careful use of transactions or other concurrency controls.

Remember that building and maintaining a system like this is a complex task that requires a strong understanding of databases, backend development, and possibly distributed

systems. It's also important to ensure that your system is secure and respects all relevant data privacy laws, especially if you're handling sensitive data or credentials.